PRINCIPIOS DE DISEÑO

CÓDIGO MALOLIENTE

- Código duplicado: Idéntico o similar código existe en más de una localización.
- Clase grande: Una clase que se ha convertido en muy grande (clase Dios).
- Clase perezosa: Una clase que hace pocas cosas.
- Clase de datos: Clases que almacenan datos, pero no comportamiento.
- Envidia de las características: Una clase que usa los métodos de otra clase en demasía.
- Intimidad inapropiada: Una clase que tiene dependencias en los detalles de implementación de otra clase.
- Rechazar el legado: Una clase sobrescribe un método de forma que el contrato de la clase base no se cumple en la clase derivada.
- Cúmulos de datos: Grupos de datos que habitualmente aparecen juntos deberían formar parte de un mismo objeto.
- Método grande: Un método que ha crecido demasiado.
- Larga lista de parámetros: Una lista larga que es difícil de leer y que hace que las llamadas a la función resulten complicadas.
- Sentencias switch: Muchas de las veces que usamos una sentencia switch debería considerarse usar polimorfismo.

DISEÑO MALOLIENTE

- Rigidez: El sistema es difícil de cambiar porque cada cambio fuerza muchos otros cambios en otras partes del sistema.
- Fragilidad: Los cambios causan que el sistema se rompa en lugares que no tienen relación conceptual con la parte que fue cambiada.
- Inmovilidad: Es difícil separar el sistema en componentes que pueden ser reutilizados en otros sistemas.
- Viscosidad: Hacer las cosas bien es más difícil que hacer las cosas mal. Los cambios que preservan el diseño son más difíciles de hacer que los trucos (hacks).
- Complejidad innecesaria: Sobreingeniería. El diseño contiene infraestructuras que no añaden ningún beneficio directo.
- Repetición innecesaria: El diseño contiene la repetición de estructuras que se podrían unificar en una sola abstracción. Cortar y pegar es una alternativa cuando editas texto, pero a la hora de editar código puede tener resultados desastrosos.
- Opacidad: Es difícil de leer y comprender. No expresa correctamente sus intenciones.

PRINCIPIOS DE DISEÑO

Guía que, cuando aplicadas conjuntamente, hacer más sencillo al programador desarrollar software que es fácil de mantener y de extender. Son recomendaciones básicas y genéricas que ayudan a los desarrolladores a eliminar los “olores” y crear un mejor diseño.

PRINCIPIOS SOLID

Single Responsibility Principle (SRP) Principio de responsabilidad única

Cada objeto debe tener una responsabilidad única que esté enteramente encapsulada en la clase. Todos los servicios que provee el objeto están estrechamente alineados con dicha responsabilidad.

@informaticapau

Una responsabilidad es un factor de cambio, si esta cambia es necesario cambiar el código que lo implementa.

Si una clase implementa una única responsabilidad entonces la clase tiene solo una razón para cambiar. Si una clase implementa más de una responsabilidad entonces las responsabilidades están acopladas. El cambio en una de ellas puede afectar a las otras, lo que conlleva un diseño frágil. Separando responsabilidades limitamos la propagación de los cambios.

Cohesión

Es a la medida en que las funciones que incluye un objeto están relacionadas funcionalmente entre sí. Mide la cantidad de trabajo que un objeto es capaz de hacer.

No es algo fácil de medir y un posible estimador sería el número de líneas de código y de métodos del objeto.

Se busca que los objetos presenten una alta cohesión porque así están más centrados en un objetivo a resolver (responsabilidad única), son más comprensibles, más manejables y, como efecto lateral, suelen presentar bajo acoplamiento.

Se busca evitar las clases Dios que lo hacen todo en un programa dejando pocas responsabilidades al resto de clases.

Open Closed Principle (OCP) Principio abierto-cerrado

Las entidades software (clases, módulos, etc.) deberían ser abiertas para permitir su extensión, pero cerradas frente a la modificación.

En otras palabras, queremos ser capaces de cambiar lo que hace una clase, sin tener que tocar el código de la clase. Esto en orientación a objetos puede hacerse fácilmente a través de los mecanismos de herencia.

Liskov Substitution Principle (LSP) Principio de sustitución de Liskov

Las clases derivadas deben ser sustituibles por sus clases base.

Una subclase siempre puede ser pasada allá donde se requiera una clase base, pero eso no garantiza que la subclase sea un subtipo conductual de la clase base.

Aquellos métodos que utilicen referencias a clases base deben ser capaces de usar objetos de clases derivadas sin saberlo.

Interface Segregation Principle (ISP) Principio de segregación de interfaces

Muchos interfaces específicos para cada cliente son mejores que un único interfaz de propósito general.

De una clase servidora se puede crear un interfaz con todos los servicios que ofrece, pero es más correcto segregar varios interfaces específicos para cada tipo de servicio en particular.

El Principio de Segregación de Interfaces indica que debemos diseñar nuestros interfaces pensando cómo serán usados y no en cómo serán implementados.

Dependency Inversion Principle (DIP) Principio de inversión de la dependencia

La inversión de la dependencia es la estrategia de depender de interfaces o clases y funciones abstractas, en vez de depender de clases y funciones concretas.

@informaticapau

Los diseños procedimentales presentan un esquema de dependencias en el que los módulos de alto nivel dependen de módulos de bajo nivel, que a su vez dependen de otros módulos de más bajo nivel, etc.

Una arquitectura orientada a objetos muestra una estructura de dependencia muy diferente, en la cual la mayoría de las dependencias apuntan hacia abstracciones. Los módulos de alto nivel no dependen de los módulos de bajo nivel, ambos dependen de abstracciones. De esta forma la dependencia entre ellos se ha invertido.

Inyección de dependencias

Una inyección es el paso de una dependencia (un servicio) a un objeto dependiente (un cliente) que podría usarlo, sin permitir que el cliente construya o encuentre dicho servicio.

TIPOS DE HERENCIA

Tipos adecuados de herencia

- **Herencia de especialización**: Las subclases sobrescriben a la superclase para ofrecer una versión más especializada de sus métodos, pero respetando las especificaciones del padre en todos los aspectos relevantes.
- **Herencia de especificación**: La clase padre define una conducta, pero no la implementa, sino que es implementada en las subclases. Se usa para garantiza que las clases mantienen un cierto interfaz común.
- **Herencia de extensión**: La subclase no sobrescribe el código de la superclase, sino que añade código nuevo. La subclase es una versión especializada de la superclase pero que no redefine su comportamiento, sino que añade un nuevo comportamiento. Como la funcionalidad del padre no ha sido alterada este tipo de herencia no incumple el principio de principio de sustitución.

Tipos inadecuados de herencia

- **Herencia de construcción**: La clase hija usa el comportamiento definido en la superclase, pero no cumple una relación “ES UN” con la clase padre. Se utiliza, desde un punto de vista pragmático, para reusar código y facilitar la creación de nuevas clases. Una alternativa más razonable sería usar composición.
- **Herencia de variación**: Ocurre cuando dos o más clases tienen implementaciones similares, pero no poseen ninguna relación jerárquica entre ellas. Arbitrariamente una ejerce de superclase del resto y el resto hereda el código común. Una mejor solución sería factorizar las características comunes en una clase padre a todas las clases.
- **Herencia de limitación**: El comportamiento de la subclase es menor o más restrictivo que el comportamiento de la superclase. Incumple explícitamente el principio de sustitución.
- **Herencia de combinación**: Ocurre cuando una clase hereda de más de una clase (herencia múltiple).

@informaticapau